

SYSTEM REQUIREMENTS SPECIFICATION

Daily RO Production Dashboard

Version 2.0

Real-Time Labor Visibility for Service & Repair Operations
Microsoft SQL Server (RO Writer) Integration

Document Type	System Requirements Specification (SRS) for Pilot & Production Deployment
Audience	RO Writer Vendor Support • Internal IT / Infrastructure • Database Administration • Service Operations Leadership
Prepared	July 2026
Classification	Internal Use Only — Confidential
Purpose	Provide complete technical requirements so IT and RO Writer can provision a secure testing environment and evaluate readiness for production rollout.

Bottom line: This is a lightweight, read-only Python web application. It needs a Windows or Linux host with Python 3.10+, Microsoft ODBC Driver for SQL Server, and a dedicated read-only SQL login that can SELECT labor data from the RO Writer database. No agents, no schema changes, no write permissions, no cloud dependency.

1. System Overview

The Daily RO Production Dashboard is a browser-based operational intelligence tool built for service and repair organizations running RO Writer. It converts raw labor time entries into immediate, actionable views of daily production, technician efficiency, hourly work patterns, and day-over-day / multi-day trends.

The application is deliberately simple: a Python Streamlit process (or optional static HTML frontend) that issues read-only SQL queries against the RO Writer Microsoft SQL Server database via pyodbc + Microsoft ODBC Driver. All aggregation, charting, filtering, and Excel generation happens in application memory. Nothing is written back to the database.

1.1 Core Capabilities

- Daily Production KPIs: Total logged hours, unique Repair Orders, average hours per RO, live variance vs comparison day
- Hours-per-RO Detail: Sortable/filterable table with technician, department/bay, hours, entry count, and % of day contribution
- Hourly Monitoring: Bar charts of work distribution by hour of day + line overlays for day-over-day pattern comparison
- Efficiency Metric: $\text{Actual Hours} \div (\text{Unique Active Technicians} \times 7 \text{ expected man-hours per day})$. Configurable standard.
- 7-Day and 30-Day Trends: Line charts tracking volume and efficiency over time
- Weekly Production Tracker: Excel-style layout with per-day Earned / Goal and Hours-to-Goal columns
- One-click multi-sheet Excel export (Executive Summary + RO detail + Hourly + Raw logs + Comparison)
- CSV extracts of any view
- Technician and Department filters (dynamic after first load)
- Ability to mark technicians as OFF for the day so they are excluded from efficiency calculations
- Full mock-data mode for demos and UI validation with zero database connection

1.2 Architecture Summary

Browser (Chrome/Edge/Firefox) → Streamlit application (Python) → pyodbc → Microsoft ODBC Driver 17/18 → RO Writer Microsoft SQL Server (read-only login). Optional static HTML frontend exists for zero-install demos but the production path is Streamlit + direct SQL.

2. Hardware Requirements

2.1 Application Server (Streamlit Host)

Component	Minimum	Recommended (Pilot / Light Production)	Notes
CPU	2 cores / vCPU	4 cores / vCPU	Modern x86_64. Hyper-threaded cores are fine.
RAM	4 GB	8 GB	Pandas + Plotly + concurrent users drive memory use. 8 GB leaves headroom.
Disk	10 GB free	20–30 GB free	App code is tiny. Space is for OS, Python, logs, temporary Excel exports, and future growth.
Network	1 Gbps NIC internal	1 Gbps NIC internal	Must reach SQL Server. Latency < 20 ms preferred.

A standard Windows Server 2019/2022 VM, a modest Linux VM, or even a dedicated Windows 10/11 workstation is sufficient for pilot and most single-site production use. No GPU required. No special hardware.

2.2 Client Workstations

Any modern Windows, macOS, or Linux desktop/laptop that can run a current browser. No client software installation is required. Tablets work for viewing but filtering and Excel export are more practical on a full keyboard/mouse setup.

3. Software Requirements

3.1 Operating System (Application Server)

OS	Support Level	Notes
Windows Server 2019 / 2022	Preferred / Primary	Easiest ODBC Driver installation and SQL Server connectivity. Recommended for most IT environments.
Windows 10 / 11 Pro	Acceptable for Pilot	Fine for testing and small deployments. Not ideal as long-term production host.
Ubuntu 22.04 LTS / 24.04 LTS	Fully Supported	Requires Microsoft ODBC Driver installation via Microsoft packages. Common in Linux-centric shops.
RHEL 8 / 9, Rocky, Alma	Supported	Same Microsoft ODBC packages. Verify SELinux policies if enforced.
macOS	Development only	Works for local testing. Not recommended as shared production host.

3.2 Python Runtime

Python **3.10, 3.11, or 3.12** is required. 3.11 is the current recommended version for stability and performance. Python 3.9 and earlier are not supported. Python 3.13 may work but has not been formally validated with the current dependency set.

Installation method is flexible: official python.org installer on Windows, deadsnakes PPA or system packages on Ubuntu, pyenv, or a container base image. A dedicated virtual environment (venv) is strongly recommended so the dashboard's packages do not collide with other Python tools on the host.

3.3 Required Python Packages

Install exactly from the project's requirements.txt. Do not guess versions. The locked minimums are:

Package	Minimum Version	Purpose
streamlit	≥ 1.28.0	Web application framework and UI
pandas	≥ 2.0.0	Data loading, aggregation, filtering, Excel generation
numpy	≥ 1.24.0	Numerical support for pandas and mock data
plotly	≥ 5.18.0	Interactive charts (hourly bars, trends, overlays)
openpyxl	≥ 3.1.0	Multi-sheet .xlsx export generation
pyodbc	≥ 5.1.0	Microsoft SQL Server connectivity
requests	≥ 2.31.0	Optional REST API path (legacy / alternative)
python-dotenv	≥ 1.0.0	Environment variable / secrets loading support

Installation command (after creating and activating a virtual environment):

```
pip install -r requirements.txt
```

3.4 Microsoft ODBC Driver for SQL Server

This is non-negotiable for the recommended direct-SQL path. The application uses pyodbc, which requires a Microsoft-supplied ODBC driver.

Platform	Driver	Installation Notes
Windows	ODBC Driver 17 or 18 for SQL Server	Usually already present on modern Windows Server / Windows 10/11. If missing, download from Microsoft.

Platform	Driver	Installation Notes
Ubuntu / Debian	msodbcsql17 or msodbcsql18	Install Microsoft's official packages (curl Microsoft key + repo, then apt-get install msodbcsql18). ACCEPT_EULA=Y is required.
RHEL / Rocky / Alma	msodbcsql17 or msodbcsql18	Microsoft provides yum/dnf packages. Follow Microsoft's Linux installation guide for the matching distro version.

Driver 18 is preferred. Driver 17 is still fully supported. Older Microsoft ODBC drivers (13.x, etc.) should be avoided.

3.5 Client Browser Requirements

Google Chrome 120+, Microsoft Edge 120+, or Mozilla Firefox 120+ (or current ESR). Internet Explorer is not supported. The dashboard uses modern JavaScript and CSS; older browsers will break charts and layout. No browser extensions or special configuration required.

4. Database Requirements (RO Writer / Microsoft SQL Server)

The dashboard does not create tables, views, or stored procedures. It only issues SELECT statements against existing labor data.

4.1 SQL Server Version

Any Microsoft SQL Server version currently supported by RO Writer is acceptable. In practice this means SQL Server 2016 and later (including SQL Server 2017, 2019, 2022, and Azure SQL Managed Instance if RO Writer is hosted that way). SQL Server Express is acceptable for testing if the data volume fits within Express limits.

4.2 Required Data Access

A dedicated **read-only** SQL login is mandatory. The login must be able to SELECT the labor / time-entry information used by RO Writer. Exact table and column names vary by RO Writer version and any customizations; the application's `fetch_data()` function contains a clear, commented example query that must be adjusted by the DBA or RO Writer specialist to match the actual schema.

Minimum logical columns the application expects after mapping:

Logical Column	Required?	Description / Usage
ro_number	Yes	Repair Order identifier. Used for unique RO counts and per-RO hour totals.
technician (or tech name)	Yes	Person who logged the time. Used for filtering, active tech count, and efficiency.
logged_hours	Yes	Hours (or fraction of hours) for that labor line. Summed for all KPIs.
log_timestamp or start_time	Strongly Preferred	Date/time the work was performed. Required for accurate hourly distribution charts. If missing, all hours are forced to hour 12.
department / bay / work center	Preferred	Used for department filters and grouping. Can be hard-coded or left null if not available.
work_date	Yes (for filtering)	Date of the labor. Used in the WHERE clause to restrict to the selected report day.

4.3 Permissions Model

- Create a SQL login specifically for this dashboard (example name: `ro_dashboard_readonly`).
- Grant only SELECT on the necessary tables or, better, on a dedicated database view that already projects the required columns.
- Do not grant INSERT, UPDATE, DELETE, EXECUTE, or any DDL rights.
- Prefer Windows Authentication if the application server is domain-joined; otherwise SQL Authentication with a strong password stored only in `secrets.toml`.

- Connection string must include TrustServerCertificate=yes or proper certificate configuration if using encrypted connections.

4.4 Network Connectivity to SQL Server

The host running the Streamlit process must be able to open a TCP connection to the SQL Server instance (default port 1433, or the custom port configured for the RO Writer instance). Firewall rules on the SQL Server host and any intermediate network devices must allow the dashboard server's IP address. Named instances require the SQL Browser service or an explicit port.

5. Security Requirements

The application is designed around a single non-negotiable principle: it is read-only. There is no code path that can modify RO Writer data.

- Credentials live only in `.streamlit/secrets.toml` (or environment variables). This file must never be committed to source control and must be readable only by the service account that runs the dashboard.
- The SQL login used by the application must be restricted to SELECT. Periodic review of that login's permissions is recommended.
- Deploy the Streamlit process on an internal network segment. Prefer placing it behind a reverse proxy (IIS, nginx, Caddy, Traefik) that can enforce authentication (Windows Integrated, OIDC, or simple basic auth) if the organization requires it.
- Streamlit's default is to listen on all interfaces on port 8501. For production, bind to localhost or an internal interface and terminate TLS and auth at the reverse proxy.
- No data is persisted by the application beyond an in-memory cache with a short TTL (default 5 minutes). Excel exports are generated on demand and live only in the user's browser download.
- All SQL is parameterized. There is no string concatenation of user-supplied values into SQL statements in the current codebase.
- Audit logging of who accessed the dashboard and which dates they queried can be added at the reverse-proxy or application level if required by policy.

Because the tool uses only an existing read-only path and never receives write credentials, the security review footprint is intentionally small compared with tools that require broader database access or ETL processes.

6. Network, Ports & Deployment Options

6.1 Ports

Port	Direction	Purpose
1433 (or custom SQL port)	Outbound from app server → SQL Server	pyodbc / ODBC connection to RO Writer database
8501 (default Streamlit)	Inbound to app server (or reverse proxy)	Browser access to the dashboard UI. Changeable via streamlit config or command line.
443 / 80	Inbound to reverse proxy (recommended)	If TLS and authentication are terminated at IIS/nginx/etc.

6.2 Deployment Patterns

Pilot / Proof-of-Concept (fastest)

Install Python + ODBC driver on a Windows workstation or small VM that already has network reach to the RO Writer SQL Server. Create a venv, install requirements, drop in secrets.toml with the read-only connection string, set `USE MOCK DEFAULT = False`, and run `streamlit run app.py`. Users point browsers at `http://<host>:8501`. Total time once the SQL login exists: usually under a day.

Production (recommended)

Run the Streamlit process under a dedicated service account (Windows Service via NSSM or systemd on Linux). Place a reverse proxy in front for HTTPS, authentication, and logging. Optionally containerize with Docker for consistency across environments. The application itself remains a single Python process; horizontal scaling is rarely needed for this workload size.

HTML Frontend Option

A static HTML + Chart.js frontend is included for pure client-side demos and for environments that want zero server-side Python. In its current form it runs fully in the browser with mock data. Connecting it to live data requires either embedding an API or using the Streamlit backend as the data source. For formal RO Writer testing the Streamlit path is the supported production route.

7. Performance Expectations

Metric	Expectation	Notes
Typical daily labor volume	50–300 rows	Application is designed around this range. Larger shops with 1 000+ daily lines still work but may benefit from a tighter SQL view.
Report generation time	< 3–5 seconds	Includes SQL round-trip + pandas aggregation + chart rendering for a normal day.
Concurrent users	5–15 interactive users	Comfortable on the recommended 4-core / 8 GB host. Streamlit is multi-session; extreme concurrency would require multiple instances or a different framework.
Cache TTL	5 minutes (configurable)	Same date queried again within the TTL returns from cache. Clearable from the UI.

8. Testing Environment Setup Checklist (for IT)

Use this as the working checklist when provisioning the test environment for RO Writer validation and internal acceptance testing.

1. Host Provisioning

Allocate a Windows Server 2019/2022 VM (or Ubuntu 22.04) with the hardware specs in Section 2.1. Join to domain if Windows Authentication will be used.

2. Python Installation

Install Python 3.11 (or 3.10/3.12). Verify with `python --version`. Create a dedicated virtual environment for the dashboard.

3. ODBC Driver

Confirm Microsoft ODBC Driver 17 or 18 is installed and visible to the system. On Linux follow Microsoft's official package instructions and accept the EULA.

4. SQL Login Creation

On the RO Writer SQL Server create a login (e.g. `ro_dashboard_readonly`). Map it to a database user with SELECT only on the required tables or a purpose-built view. Document the exact connection string.

5. Firewall / Network

Open the SQL port from the dashboard host IP to the SQL Server. Test connectivity with a simple `pyodbc` or `sqlcmd` connection from the dashboard host before deploying the app.

6. Application Deployment

Copy the `ro_production_dashboard` folder to the host. Inside the activated venv run `pip install -r requirements.txt`. Create `.streamlit/secrets.toml` containing the `[mssql]` `connection_string`.

7. Configuration Toggle

In `app.py` set `USE MOCK_DEFAULT = False`. Adjust the example SQL query inside `fetch_data()` so column and table names match the actual RO Writer schema. Keep the query strictly SELECT.

8. First Smoke Test

Run `streamlit run app.py`. Open the browser, leave mock mode off, pick a known good date that contains labor data, and click Generate Report. Confirm KPIs, tables, and charts populate.

9. Security Validation

Confirm the SQL login cannot write. Confirm `secrets.toml` permissions are restricted. Confirm the Streamlit process is not exposed to the public internet.

10. User Acceptance

Hand the URL to 2–3 service managers. Have them exercise date selection, filters, efficiency calculation, hourly charts, and Excel export. Collect feedback on any missing columns or desired filters.

9. Assumptions, Constraints & Open Items

Assumptions

- RO Writer's underlying Microsoft SQL Server instance is reachable from the chosen application host.
- Labor data exists at a granularity that includes (or can be derived into) RO number, technician, hours, and a usable timestamp or hour-of-day.
- A 7-hour expected productive day per technician is an acceptable organizational standard for the efficiency metric. This constant is trivial to change in code if a different number is preferred.
- Users will access the tool from the internal network or via an already-approved remote-access method (VPN, etc.).
- IT will supply or approve the exact SQL query / view that maps RO Writer tables to the logical columns listed in Section 4.2.

Constraints

- The application performs no schema changes and will not request elevated SQL privileges.
- No agent or scheduled job is installed on the SQL Server itself.
- The current HTML frontend is not the production data path; live testing should use the Streamlit application.
- Historical trend depth is limited by whatever date range the underlying labor tables retain and by the performance of the SELECT statements against those tables.

Open Items Requiring RO Writer / IT Input

- Exact table and column names (or preferred view definition) for labor time entries.
- Confirmation of whether a single `work_date + start_time` model or a different time-entry structure is used.
- Preferred authentication method for the SQL login (Windows vs SQL auth).
- Any existing read-only reporting accounts that can be reused or cloned.
- Network path and firewall ownership between the proposed dashboard host and the SQL Server.

10. Recommended Immediate Next Steps

1. IT schedules a short working session with the RO Writer DBA or support contact to identify the correct labor tables and produce a SELECT-only view or documented query.
2. IT provisions the test host and the read-only SQL login.
3. Dashboard owner deploys the application in mock mode first so stakeholders can see the full UI without waiting on data connectivity.
4. Once the SQL connection works, run a side-by-side validation: pick 2–3 historical dates, generate the dashboard report, and compare totals against known RO Writer reports or existing Excel extracts.
5. After successful validation, expand the pilot to one high-volume service location for 2–4 weeks of real daily use before any broader rollout.

This document describes a complete, self-contained, read-only solution. There are no hidden dependencies, no cloud services, and no requirement to modify the RO Writer application or its database schema. The only things IT and RO Writer need to supply are network reachability and a properly scoped SELECT login.

Questions about the application code, the example SQL, or customization of the efficiency formula should be directed to the internal owner of this dashboard. Questions about RO Writer table structures and SQL Server access should be directed to the RO Writer support channel and the local DBA team.